

Студент: Чернышева Елизавета Сергеевна  
Курс: 3  
Группа: Z33433  
2025 г.

# Лабораторная работа №3

## C++ CLI / FUNCTION / LOOP / RECURSION

### 1 Цель работы

Познакомить студента с основными алгоритмическими конструкциями, которые будут использоваться для создания CLI программы. Далее продемонстрировать эффективность использования механизма рекурсии.

C++ алгоритмы: CLI Калькулятор вещественных чисел  $+$ ,  $-$ ,  $^$ ,  $.$  Реализация с использованием только функций, условий, циклов,  $+$  и  $-$ . Вид команд в консоли: `calc / plus / minus / power`; Ханойская башня, результат корректной последовательности.

### 2 Задачи

- [C++ CLI CALC] Создать программу CALC с интерфейсом CLI**  
Создать программу под названием CALC, которая будет принимать на вход 3 аргумента (2 операнда и 1 оператор). Оператор может быть:  $+$ ,  $-$ ,  $^$ ,  $.$  Реализация операторов только с использованием функций, условий, циклов,  $+$ ,  $-$  и  $*$ .
- [C++ RECURSION] Решить задачу ханойской башни с использованием рекурсии**  
Описание: Ханойская башня является одной из популярных головоломок XIX века. Даны три стержня, на один из которых нанизаны восемь колец, причём кольца отличаются размером и лежат меньшее на большем. Задача состоит в том, чтобы перенести пирамиду из восьми колец за наименьшее число ходов на другой стержень. За один раз разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее. Результат обнаруженной последовательности шагов записать в виде двусвязного списка. В конце программы сделать вывод этого списка на экран. Освободить память списка перед завершением программы.
- [SAVE] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)**  
Фиксацию ревизий производить строго через ветку `dev`. С помощью скриптов накатить ревизии на `stg` и на `prd`. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

### 3 Решение

#### 1. [C++ CLI CALC] Создать программу CALC с интерфейсом CLI

В папке `src` создаем отдельный файл `power_func.cpp` с функцией `power()`, которая реализует возведение числа в степень. Она понадобится, поскольку по условию задания мы не можем воспользоваться встроенной функцией `pow()`, а только лишь собственными функциями, условиями, циклами и операторами `+`, `-`, `^`. Так же в папке `inc` создаем хедер файл `power_func.h`. К нему нужно будет обратиться из файла `.cpp` с основной функцией `main()`.

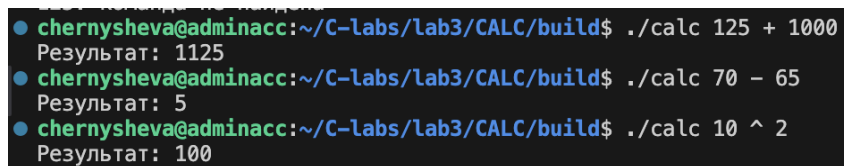
В папке `src` хранится основной файл `calc.cpp`, в котором производится проверка количества вводимых аргументов, их парсинг, выполняются запрашиваемые операции и выводится результат. Если аргументов будет не 3, программа выдаст ошибку. При этом если операнды будут содержать много цифр, ошибки не будет, так как даже многозначное число будет считаться за 1 аргумент. Также если вводимый оператор это не `+`, `-` или `^` программа так же выдаст ошибку. В программе не реализован учет пробелов, поэтому для того, чтобы программа заработала, ввод должен быть, например, `<5 + 3>`, а не `<5+3>`. Но пробелы можно учесть при необходимости.

Так как проект CALC содержит несколько `.cpp` файлов, мы собираем его с помощью `cmake`:

```
<cd build>
<cmake ..>
<make>
<./calc 5 + 3>.
```

Но для этого также понадобится `CMakeLists.txt` файл в корне проекта CALC. В нем прописываются все необходимые `.cpp` файлы, а также название программы. В нашем случае это `calc`.

Пример вывода:



```
● chernysheva@adminacc:~/C-labs/lab3/CALC/build$ ./calc 125 + 1000
Результат: 1125
● chernysheva@adminacc:~/C-labs/lab3/CALC/build$ ./calc 70 - 65
Результат: 5
● chernysheva@adminacc:~/C-labs/lab3/CALC/build$ ./calc 10 ^ 2
Результат: 100
```

#### 2. [C++ RECURSION] Решить задачу ханойской башни с использованием рекурсии

Создаем в `src` файл `towerOfHanoi.cpp`, который содержит функцию для решения задачи Ханойской башни через рекурсию. Проще всего решать задачу с конца - чтобы переложить `n` дисков на свободную ось, нужно переложить на нее последний самый большой диск, а это можно сделать, когда остальные `n-1` диск находятся на другой оси. То есть решение получается таким:

1. Перекладывание стека из `n-1` дисков на независимую ось;
2. Перекладывание `n`-го диска на нужную нам ось;
3. Перекладывание стека из `n-1` дисков на нужную нам ось.

Записанный ход решений мы оформляем в виде двусвязного списка `std::list<string>`. Также

создаем towerOfHanoi.h файл для обращения к функции из main().

Создаем main.cpp файл, где задается число дисков, освобождается место под двусвязный список в куче (оперативной памяти), сохраняется туда список, выводится и далее освобождается память в куче, выделенная под список.

Проект состоит из нескольких .cpp файлов, поэтому, как и в задаче 1) создаем CMakeLists.txt, который содержит исполняемый файл под названием hanoi\_tower. И собираем проект с помощью cmake из папки build. Запускаем проект: `<./hanoi_tower>`. Пример вывода для N=3 дисков:

```
● chernysheva@adminacc:~/C-labs/Lab3/HANOITOWER/build$ ./hanoi_tower
Move disk 1 from rod A to rod C
Move disk 2 from rod A to rod B
Move disk 1 from rod C to rod B
Move disk 3 from rod A to rod C
Move disk 1 from rod B to rod A
Move disk 2 from rod B to rod C
Move disk 1 from rod A to rod C
```